

# Платформа Converged AI

**Converged AI** — open-source платформа для производственных компаний: сервисных бюро 3D-печати, CNC-мастерских, R&D-лабораторий и сетей небольших производств.

Её задача — забрать на себя всё, что происходит вокруг производства: заявки, файлы, расчёты, очереди, статусы заказов, коммуникацию с клиентами, оборудование, оплату, доставку и повторные продажи. Станки продолжают делать детали, команда продолжает принимать инженерные решения, а Converged связывает это в одну управляемую систему.

Внутри платформы есть **17 решений**, сгруппированных в четыре направления: заказы и клиенты, производство и запасы, деньги и прибыль, команда и ответственность. Это не отдельные «модули ради модулей», а готовые сценарии под конкретные проблемы мастерской.

# Что делает Converged

## Что делает Converged

---

У производственной компании есть два слоя работы. Первый — изготовить деталь, собрать изделие, выполнить заказ. Второй — всё, что должно произойти до, во время и после производства: принять запрос, не потерять файл, согласовать цену, поставить задачу в очередь, предупредить клиента, понять загрузку станков, увидеть просрочку и получить оплату.

Converged закрывает второй слой. Это цифровой контур, который соединяет сайт, клиентские заявки, внутренние задачи, оборудование, склад, оплату, уведомления и аналитику. Владелец видит не набор разрозненных инструментов, а живую картину бизнеса: что пришло, что принято в работу, что стоит в очереди, что уже готово, где риск задержки и где теряется маржа.

Платформа особенно полезна там, где производство уже работает, но управление держится на чатах, таблицах, памяти сотрудников и ручном контроле. Converged не заставляет сразу внедрять тяжёлую ERP или MES. Он начинает с конкретных процессов и постепенно собирает вокруг них единую систему.

В типовом сценарии клиент оставляет заявку через сайт, форму, мессенджер или оператора. Converged сохраняет запрос, прикрепляет файлы, создаёт заказ, помогает оценить стоимость, ставит работу в очередь, отслеживает исполнение и держит клиента в курсе. Оператор может спросить статус в интерфейсе или через ИИ-чат, а система отвечает по реальным данным, а не по догадкам.

В результате мастерская меньше зависит от ручной координации. Люди занимаются производством и решениями, которые действительно требуют опыта, а платформа берёт на себя маршрутизацию, контроль сроков, повторяющиеся сообщения, сбор статусов и подготовку действий.

# Система решений

## Система решений

---

Converged не продаётся как пустая платформа, где клиенту нужно сначала самому придумать архитектуру и собрать модули. Базовая единица ценности здесь — **решение**: готовый рабочий сценарий, который закрывает одну понятную проблему бизнеса.

Всего в платформе предусмотрено **17 решений**. Они сгруппированы в четыре направления:

- **Заказы и клиенты** — входящие заявки, витрина услуг, история клиента, статусы, коммуникация и повторные продажи.
- **Производство и запасы** — загрузка оборудования, очереди, материалы, контроль качества, сбои и отгрузки.
- **Деньги и прибыль** — себестоимость, маржа, платежи, дебиторка, ценообразование и сценарии роста.
- **Команда и ответственность** — зоны ответственности, смены, стандарты, база знаний и ввод новых сотрудников.

Решение внутри Converged — это не только экран в интерфейсе. Обычно оно включает модель данных, workflow, роли, уведомления, действия ИИ-агента и интеграции с оборудованием или внешними сервисами. Поэтому владелец выбирает не «функцию», а проблему: ускорить обработку заявок, видеть очередь станков, понять прибыльность заказов или навести порядок в сменах.

Подробное описание всех решений лучше выносить в отдельный раздел. В продуктовой документации важно зафиксировать принцип: Converged AI — это платформа, а решения — прикладные сценарии, которые работают внутри неё и постепенно закрывают разные зоны производственного бизнеса.

# Оборудование и цех

## Оборудование и цех

---

Converged не заменяет прошивку станка и не пытается управлять производством вслепую. Он подключается поверх оборудования, читает телеметрию, связывает состояние машин с заказами и показывает, что реально происходит в цехе.

Платформа рассчитана на разные типы оборудования: 3D-принтеры Bambu Lab, Marlin и Klipper, станки с ЧПУ, роботизированные узлы и специализированные адаптеры. Где возможно, Converged получает статусы, ошибки, прогресс выполнения, температуру, очередь задач и другие технические данные. Где прямое управление рискованно или недоступно, система остаётся слоем наблюдения и координации.

Для владельца это означает простую вещь: оборудование перестаёт быть набором отдельных окон. Видно, какие машины заняты, где простой, что опаздывает, какой заказ связан с конкретной операцией и где оператору нужно вмешаться. Если парк растёт, эта видимость становится важнее, чем ручное переключение между интерфейсами отдельных принтеров или станков.

Converged также связывает оборудование с бизнес-процессом. Задача не просто «печатается» или «фрезеруется» — она находится в контексте заказа, срока, клиента, материала, оплаты и следующего шага. Поэтому цех становится частью общей системы, а не отдельным островом.

# Процессы

## Процессы

---

Главная проблема растущей мастерской редко в том, что «нет ещё одной кнопки». Чаще проблема в том, что процессы живут в головах сотрудников: кто должен ответить клиенту, когда считать стоимость, кто проверяет файл, когда запускать производство, кому писать при задержке и что делать после отгрузки.

В Converged эти цепочки описываются как workflow. Типовой процесс может идти от заявки до расчёта, согласования, постановки в очередь, производства, контроля качества, оплаты, доставки и уведомлений. Пользователь обычно не собирает граф с нуля: готовые сценарии уже поставляются вместе с решениями, а настройка сводится к правилам, ролям, срокам, интеграциям и уведомлениям.

Технически исполнение вынесено в Runtime-слой. Он запускает workflow, cron-задачи, интеграционные шаги и бизнес-логику, оставаясь stateless: постоянные данные хранятся в микросервисах, а Runtime отвечает за выполнение цепочек. Такой подход не размазывает бизнес-логику по десяткам сервисов и оставляет одно понятное место, где живут правила процесса.

Для сложных внедрений workflow можно расширять. Разработчик описывает сценарии как типизированные TypeScript-классы, а ИИ-агенты могут запускать разрешённые действия внутри этих сценариев. Но для обычного пользователя цель другая: не строить конструктор, а включить готовый процесс и получить управляемый результат.

# ИИ-слой

## ИИ-слой

---

ИИ в Converged — не отдельный чат «для красоты», а слой управления поверх данных, интерфейса и процессов. Пользователь может спросить, что происходит с заказом, попросить подготовить ответ клиенту, найти задержку, запустить разрешённый workflow или собрать сводку по загрузке.

Модель получает контекст из платформы: заявки, статусы, файлы, телеметрию, историю клиента, права доступа и текущие задачи. Поэтому ответ строится не на общих рассуждениях, а на данных компании. Если нужно действие, ИИ не обходит систему напрямую: он вызывает разрешённые функции, микросервисы или workflow через контролируемый слой.

Поставщики моделей подключаются через адаптеры. В одной установке могут использоваться GPT, Claude, DeepSeek, Mistral, Gemini или другие движки, если они подходят под задачу и политику клиента. Одну модель можно использовать для общения с клиентами, другую — для разбора технических заданий, третью — для анализа документов или производственных статусов.

Ключевой принцип — управляемость. У ИИ есть собственный профиль доступа, все действия логируются, а критичные операции выполняются в рамках тех же прав и политик, что и действия человека. Это позволяет доверять агентам рутину, не отдавая им неконтролируемую власть над производством.

# Архитектура

## Архитектура

---

Converged спроектирован как модульная платформа, но не как хаотичный набор микросервисов. Разделение простое: интерфейс показывает и запускает действия, Runtime исполняет процессы, микросервисы владеют данными, адаптеры подключают оборудование и внешние системы.

**Пользователь / клиент**      ↓      **UI и микрофроненды**      ↓      **Runtime:**  
**workflow, cron, интеграции, ИИ-действия**      ↓      **Микросервисы:**  
**типизированные API и собственные данные**      ↓      **Storage / Behemoth /**  
**файлы / SQL / KV / метрики**      ↓      **Оборудование, мессенджеры, платёжные**  
**и внешние сервисы**

Микросервисы намеренно остаются тонкими. Каждый сервис отвечает за свою область данных, валидацию и типизированный API. Он не должен знать внутреннюю логику соседних сервисов и не должен превращаться в скрытый центр бизнес-процессов. Это снижает связность и делает систему проще для сопровождения.

Вся сквозная логика вынесена в Runtime. Если нужно принять заказ, запросить несколько сервисов, создать задачу, запустить уведомление, дождаться события и обновить статус, это исполняется в workflow. Runtime не хранит постоянное состояние у себя: историю, переменные и результаты он записывает через сервисы, которые владеют своими хранилищами.

Хранилище строится вокруг изоляции. Вместо одной общей базы каждый домен получает собственные границы данных: SQL, key-value, файловое хранилище, колоночные данные, векторные индексы или графовые связи там, где это нужно. Такой подход помогает переносить рабочие пространства, ограничивать доступ и избегать общей базы, в которой смешаны данные разных клиентов.

Фронтенд также модульный. Общая оболочка загружает независимые микрофроненды через import map, поэтому отдельные части интерфейса

можно развивать без полной пересборки всего продукта. Для пользователя это остаётся одной системой, а для разработки — набором независимых зон ответственности.

# Развёртывание

## Развёртывание

---

Converged рассчитан на несколько сценариев установки: от небольшой мастерской до production-развёртывания в инфраструктуре компании. Базовая платформа разворачивается поверх **k3s** — лёгкого Kubernetes, который подходит для edge-устройств, локальных серверов и облачных сред.

Главных профиля два:

- **Mono** — UI, Runtime, микросервисы, storage и cache собраны компактно. Это режим для разработки, прототипов, демонстраций и небольших установок, где важнее простота запуска.
- **Multi** — UI, Runtime-группы, доменные группы микросервисов, storage и cache разнесены. Это стандартный production-профиль, где нужны изоляция, масштабирование и более точное управление нагрузкой.

Оба профиля используют один и тот же код. Отличается только топология контейнеров и конфигурация. Поэтому можно начать с компактной установки, а затем перенести ту же систему в более серьёзную инфраструктуру без переписывания продукта.

Для self-hosted-сценария клиент контролирует установку, сеть, бэкапы, обновления и физическое расположение данных. Это подходит компаниям с внутренними требованиями к безопасности или желанием держать производство полностью у себя. Облачная поставка снимает операционные задачи: платформа разворачивается и обновляется командой сервиса, а клиент получает готовый рабочий контур.

Гибридный вариант также возможен: чувствительные данные и оборудование остаются локально, а облако используется для обновлений, внешнего доступа, координации распределённых команд или отдельных AI-функций. Важный принцип — не привязывать клиента к одному способу поставки.

# Технологии

## Технологии

---

Серверная часть Converged построена на **Bun** и **Elysia**. Bun быстро запускает JavaScript и TypeScript, экономно расходует память и хорошо подходит для компактных edge-установок. Elysia используется как HTTP-слой для backend-плагинов и микросервисов.

Контракты между сервисами описываются типизированно. NRPC связывает TypeScript-интерфейсы с реализациями и генерирует клиентские пакеты, чтобы frontend, Runtime и backend работали с одними и теми же контрактами, а не с разрозненными строковыми API.

Для хранения данных используется набор лёгких хранилищ под разные задачи: SQL, key-value, файлы, колоночные данные, векторные индексы и графовые связи. Нативный слой Behemoth и Zig-адаптеры закрывают задачи, где важны низкие накладные расходы, работа с оборудованием, Unix-сокеты и FFI.

Фронтенд построен как React-платформа с микрофронтендами. Общая оболочка загружает отдельные UI-модули, а продуктовые сценарии могут развиваться независимо. Это важно для платформы с большим числом решений: интерфейс не должен превращаться в один тяжёлый монолит.

Оркестрация и поставка строятся вокруг k3s, Helm и конфигурационных профилей. Один и тот же набор компонентов может быть собран в компактный моно-профиль или разнесён по группам для production.

# Производительность

## Производительность

---

Converged проектируется под производственные площадки, где не всегда есть большой серверный парк. Поэтому система избегает лишней тяжести: Bun снижает накладные расходы backend-процессов, Runtime остаётся stateless, а микросервисы можно группировать по типам нагрузки вместо запуска сотен отдельных контейнеров.

Производительность здесь достигается не одним трюком, а архитектурой. Данные не гоняются через лишние уровни, сервисы владеют своими хранилищами, Runtime распараллеливает workflow и cron-задачи, а нативные адаптеры используются там, где HTTP или обычный JS-слой дали бы слишком большой overhead.

Компактная установка может работать на небольшом сервере или single-board computer, если нагрузка соответствует масштабу мастерской. При росте можно разнести Runtime, микросервисы и storage-группы, чтобы использовать больше CPU-ядер, изолировать тяжёлые задачи и не останавливать всю систему из-за одного узкого места.

Важно, что платформа не обещает бесконечную производительность «из коробки». Узкие места зависят от оборудования, объёма файлов, числа заказов, AI-провайдеров и интеграций. Архитектура Converged даёт возможность начинать компактно и масштабировать только те части, которые действительно стали горячими.

# Безопасность

## Безопасность

---

Converged исходит из того, что производственные данные нельзя складывать в одну общую кучу. Заказы, файлы клиентов, технологические параметры, платежи, переписка и телеметрия оборудования должны быть отделены по рабочим пространствам и зонам ответственности.

Архитектурно это поддерживается изоляцией данных. Микросервисы владеют своими хранилищами, рабочие пространства могут иметь отдельные каталоги, ключи, файлы и границы доступа. Такой подход упрощает экспорт, перенос на self-hosted, резервное копирование и аудит.

Права доступа применяются не только к людям, но и к ИИ-агентам. Если модель запускает действие, читает данные или вызывает workflow, это должно происходить в рамках её профиля разрешений. Действия логируются, поэтому можно восстановить, кто или какой агент инициировал шаг, какие данные были затронуты и чем завершился сценарий.

Self-hosted и private-развёртывания дают клиенту полный контроль над инфраструктурой: сетью, секретами, API-ключами, бэкапами и физическим расположением данных. Облачный режим удобнее операционно, но не должен превращаться в vendor lock-in: данные должны оставаться переносимыми, а сценарии — воспроизводимыми в другой установке.

# Лицензирование

## Лицензирование

---

Converged распространяется под **AGPL-3.0**. Это копилефт-лицензия для сетевого программного обеспечения: если вы модифицируете платформу и предоставляете к ней доступ через сеть, изменения должны быть раскрыты в соответствии с условиями лицензии.

Для пользователей это означает, что self-hosted-версию можно разворачивать без покупки лицензии на сам код. Такой путь подходит компаниям, которым нужен контроль, локальная установка, аудит или эксперименты на собственном железе. Ответственность за эксплуатацию в этом случае остаётся у владельца установки: обновления, резервное копирование, мониторинг, безопасность и доступность.

Облачная поставка — это не продажа закрытого кода, а сервис вокруг open-source платформы. Клиент платит за быстрый запуск, поддержку, обновления, бэкапы, мониторинг, инфраструктуру и предсказуемую работу. Для многих мастерских это дешевле, чем собирать DevOps-компетенции внутри.

Такой баланс важен для доверия: ядро остаётся открытым, сообщество может проверять и развивать платформу, а коммерческая модель строится вокруг внедрения, сопровождения и управляемой поставки ценности.

# Сообщество

## Сообщество

---

Converged задуман как открытая производственная платформа, а не как закрытая SaaS-коробка. Базовое ядро доступно под open-source лицензией, а вокруг него могут появляться интеграции, микросервисы, микрофронтенды, workflow и прикладные решения.

Это важно для производственного рынка: у разных мастерских разное оборудование, разные материалы, разные стандарты качества и разные цепочки поставок. Закрытая система быстро упирается в границы конкретного вендора. Открытая архитектура позволяет добавлять адаптеры и сценарии под реальные условия.

Расширение может быть простым: новый адаптер оборудования, интеграция с платёжным сервисом, импорт из старого сайта, отдельный UI-модуль или workflow для конкретной отрасли. Если расширение полезно другим участникам, оно может стать частью каталога решений или использоваться как частное внедрение.

При этом открытость не означает отсутствие контроля. Расширения должны проходить техническую проверку, не ломать архитектурные границы и не получать доступ к данным шире, чем требуется. Доверие к экосистеме строится на исходниках, воспроизводимости и понятной модели разрешений.

# Исходные коды

## Исходные коды

---

Проект развивается открыто. Исходный код, текущая архитектура и рабочие материалы доступны в репозитории:

<https://github.com/solenopsys/converged>

Репозиторий полезен не только разработчикам. По нему можно понять, как устроены микросервисы, Runtime, микрофронтенды, генерация контрактов, профили развёртывания и аппаратные адаптеры. Для компаний, которые рассматривают self-hosted или private-внедрение, это важная часть проверки: платформа не является закрытой коробкой.